COSC204 Computer Systems

Lecture 14: Processes and system calls

Zhiyi Huang

School of Computing, University of Otago

Overview

Last lecture:

- Introduction to operating systems
- History of operating systems

This lecture:

- How operating systems manage processes
- System calls
- Interrupt handling
- Life cycle of processes

Information OS kernel stores about a process

Recall: a process is not just a program: it is a *dynamic* entity, a program in execution within the RAM of a computer.

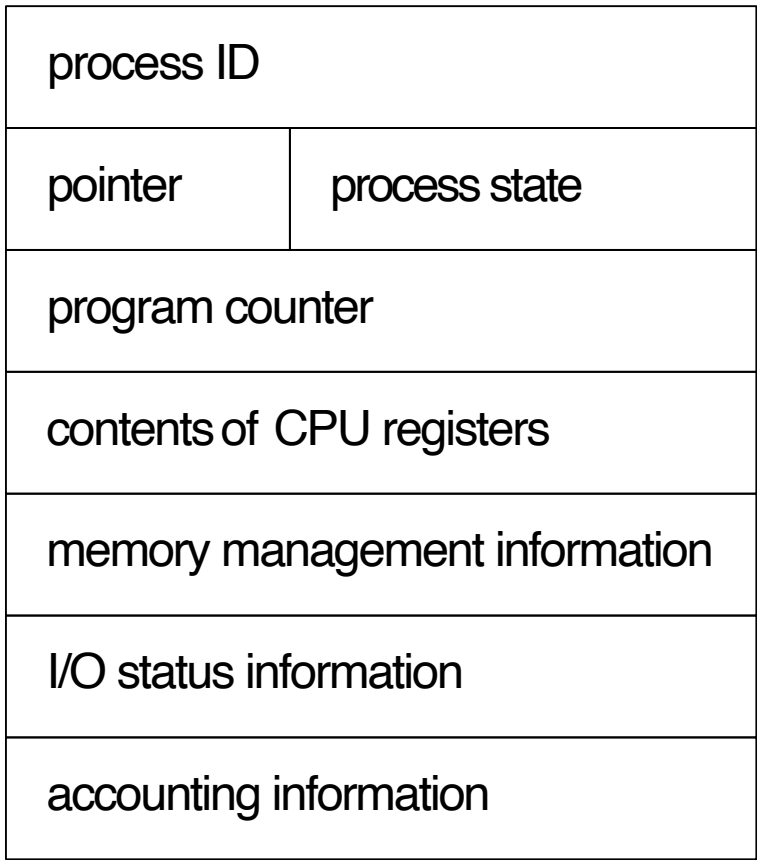
A given program (e.g. *Microsoft Minesweeper*) could be executing many times on a given machine: the machine must represent *each execution* as a separate (OS) process.

Components of an operating system process:

- text section/code section: the program code itself
- data section: any global variables used by the program
- process stack: any local variables currently being used
- program counter: a pointer to current position within program code
- contents of CPU registers
- memory management information
- device/file allocation information
- accounting information

Process Control Blocks (PCBs)

The operating system uses a data structure called the process control block (PCB) to keep a record of a process running in the system.



Processes in Linux

```
dme@rpi:~ $ ps -u dme ua
USER   PID %CPU %MEM   VSZ  RSS TTY   STAT START TIME COMMAND
root   512  0.0  0.7  4464 1320 tty1  Ss+  Aug14 0:00 /sbin/agetty -o
root   513  0.0  0.7  6632 1384 ?     Ss+  Aug14 0:00 /sbin/agetty -o
dme  22006 31.7  4.1 14356 7440 ?     Ss   23:34 0:01 /lib/systemd/sy
dme  22007  0.0  1.6 37172 3064 ?     S    23:34 0:00 (sd-pam)
dme  22024  1.0  2.3 14500 4264 ?     S    23:34 0:00 sshd: dme@pts/0
dme  22025 25.5  2.0  8444 3676 pts/0 Ss   23:34 0:00 -bash
dme  22036  0.0  1.4 10840 2588 pts/0 R+   23:34 0:00 ps -u dme ua
```

For your interest (don't memorise this detail—just look it up if/when you need it):

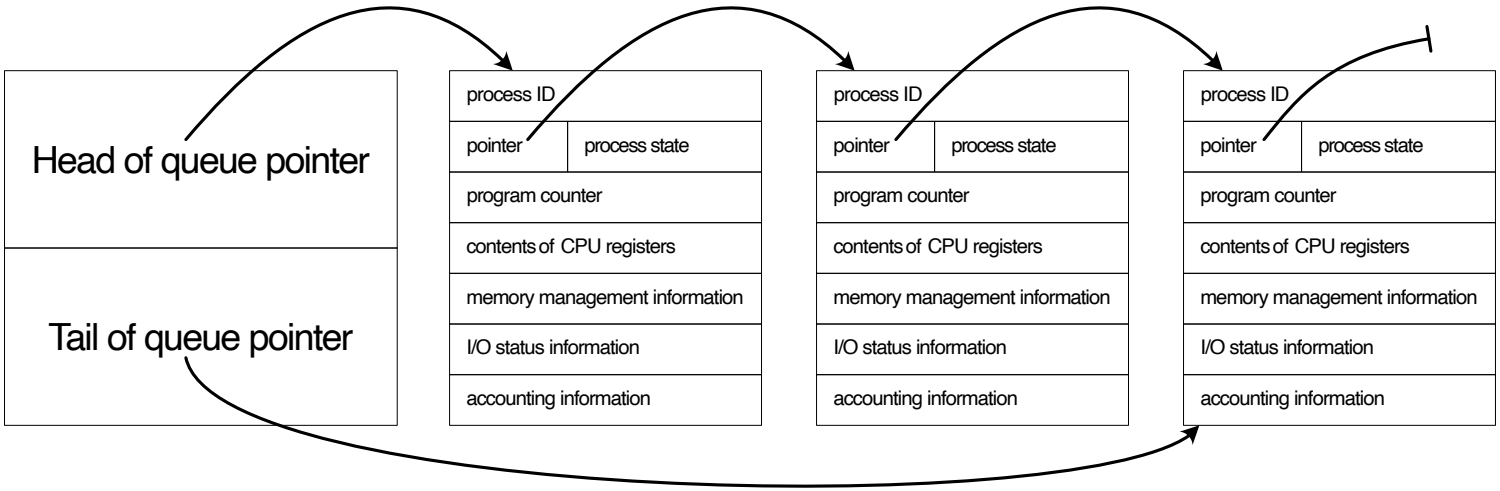
- User ID (UID): the user who owns the process (`USER`)
- Process ID (PID): a unique identifier for each process (`PID`)
- CPU usage: the percentage of CPU time used by the process (`%CPU`)
- Memory usage: the amount of RAM used by the process (`%MEM`)
- Status: the current state of the process (e.g. running, sleeping) (`STAT`)
- CPU time: the total, accumulated CPU time used by the process (`TIME`)
- Command: the command used to start the process (`COMMAND`)

Scheduling processes

Processes take turns to run on the CPU. The processes that are not running are put into *queues*. Example types of queues include:

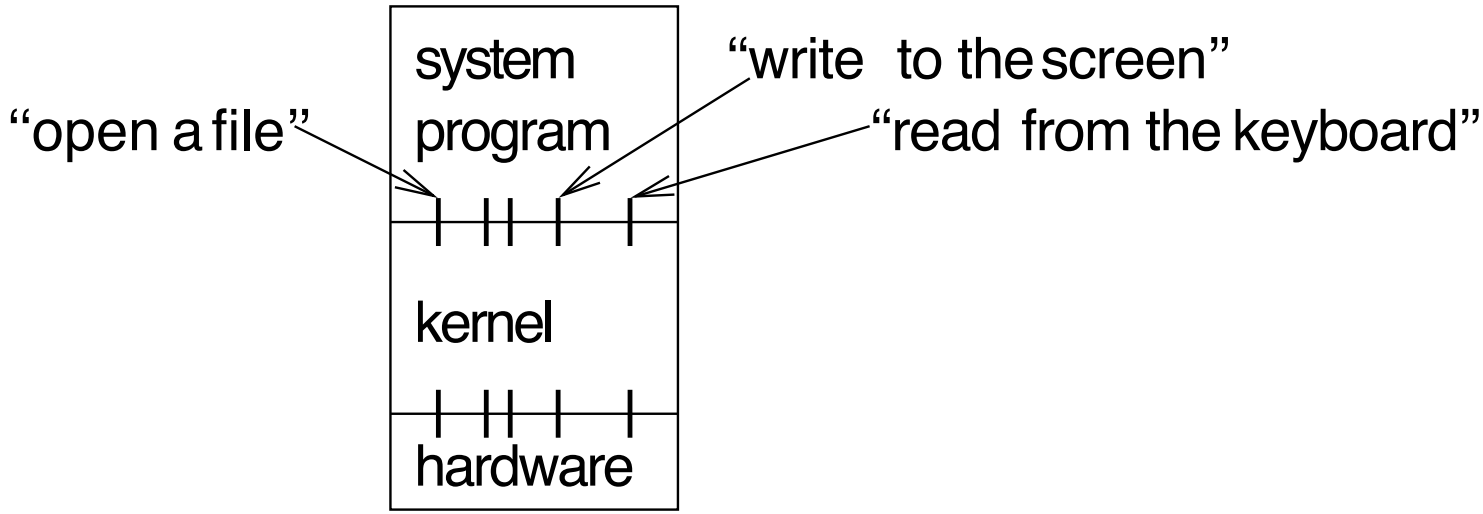
- Ready queue: all the processes that are ready to execute.
- Device queue: all the processes waiting to use a particular device. (One queue per device.)

Queues are generally represented as linked lists.



Requesting service to the kernel: system calls

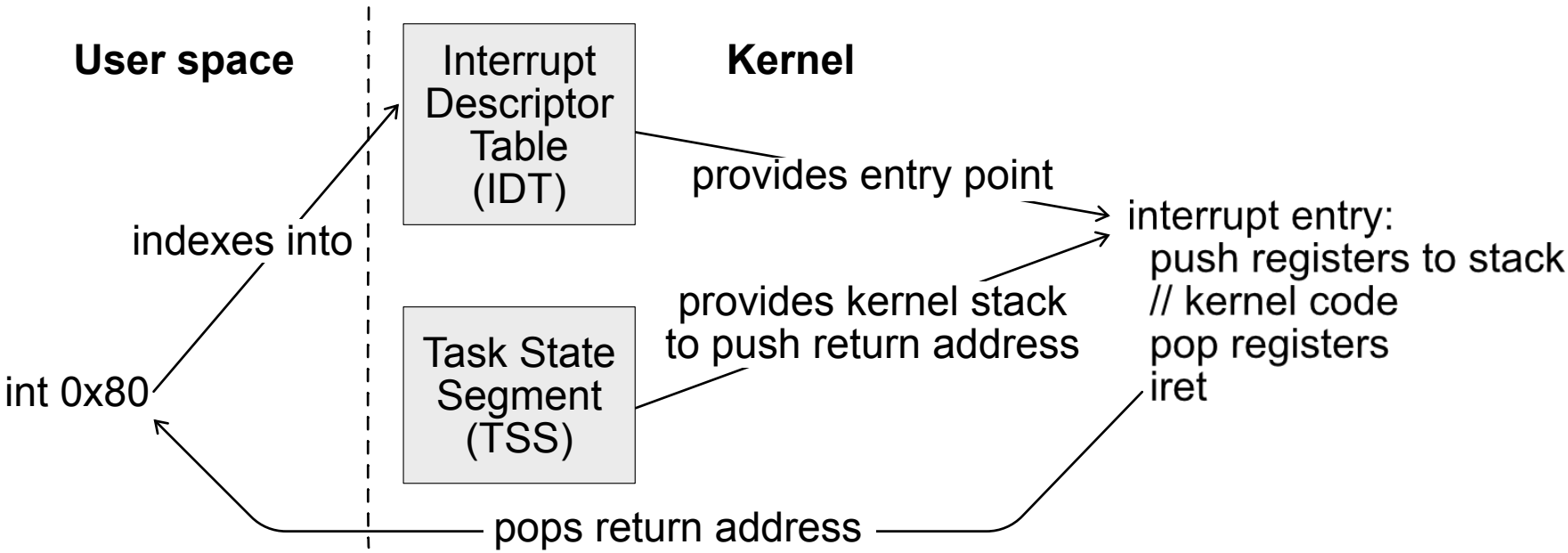
Processes issue requests to the kernel by making system calls.



System calls are requests for services to OS kernel, implemented via special instructions e.g. SYSCALL, INT, or SVC. These machine code instructions cause interrupts within the CPU.

The set of system calls is termed the programmer interface to the OS.

Linux system call execution on Intel CPUs



System calls in xv6 (1 of 2)

- `fork()` Create a new process
- `exit()` Terminate the current process
- `wait()` Wait for a child process to exit
- `kill(pid)` Terminate a process identified with *pid*
- `getpid()` Return the ID number of the current process
- `sleep(n)` Put the current process into sleep for *n* seconds
- `exec(filename, *argv)` Load a file named *filename* and execute it with arguments in *argv*
- `sbrk(n)` Update the process memory space by *n* bytes
- `open(filename, flags)` Open a file named *filename* with *flags* indicating read/write
- `read(fd, buf, n)` Read *n* bytes from an open file *fd* into *buf*
- `write(fd, buf, n)` Write *n* bytes from *buf* to an open file *fd*
- `close(fd)` Close the open file *fd*

System calls in xv6 (2 of 2)

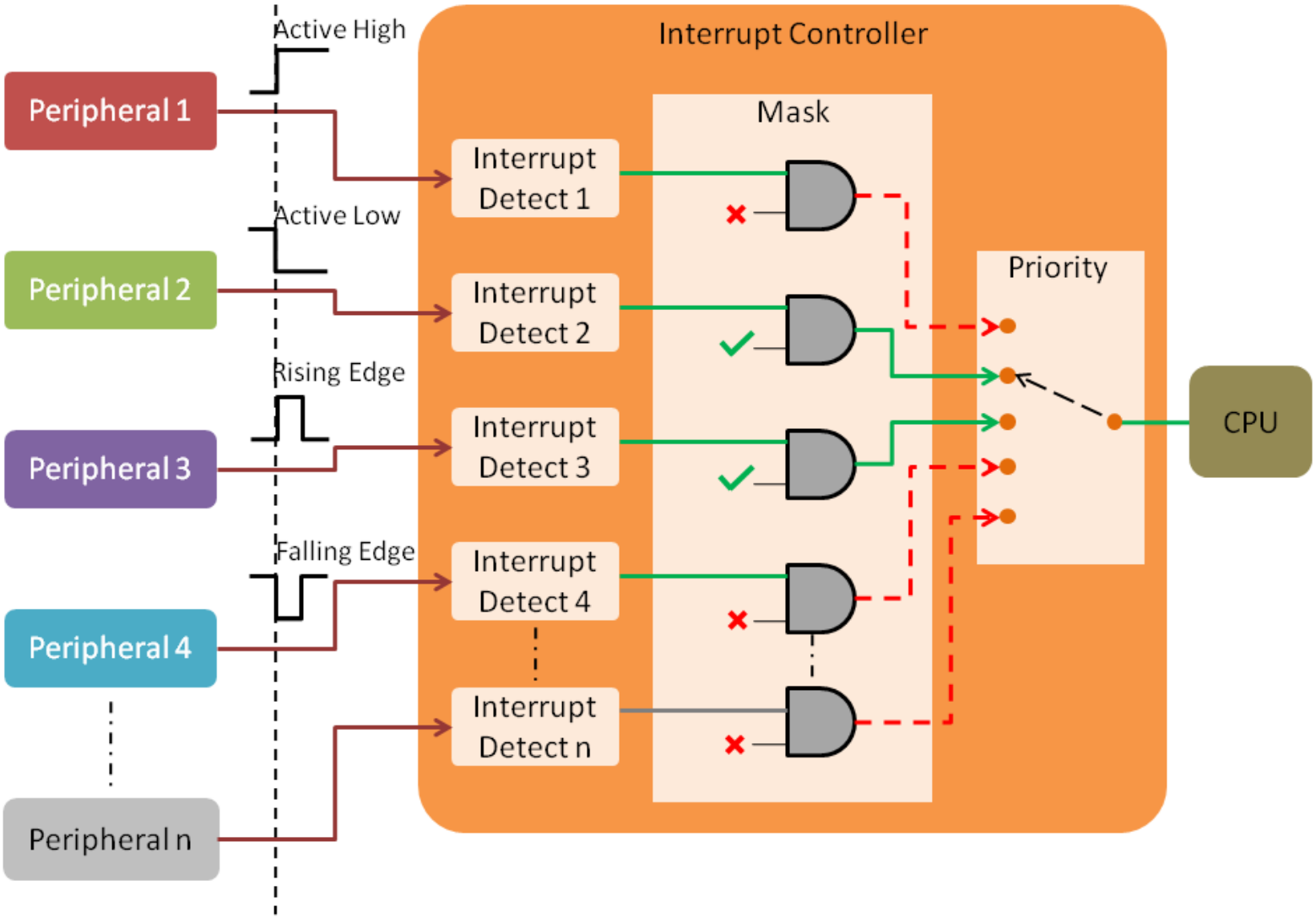
- `dup(fd)` Duplicate the open file *fd* into another file descriptor (the return value)
- `pipe(p)` Create a pipe and return the file descriptors in *p*
- `chdir(dirname)` Change the current working directory of the process to *dirname*
- `mkdir(dirname)` Create a new directory *dirname*
- `mknod(name, major, minor)` Create a device file *name* with *major* and *minor* numbers
- `fstat(fd)` Return status information about an open file *fd*
- `link(f1, f2)` Create another name *f2* for the file *f1*
- `unlink(filename)` Remove a file named *filename*

System calls are typically triggered by interrupts

- The CPU hardware contains a wire called the interrupt request line. Changing voltage on this line can be used by external hardware like keyboards to cause an interrupt of the CPU so that the OS pays attention. CPU instructions can also cause interrupts.

- The CPU runs a repeated instruction-execution cycle:
  - Fetch an instruction from memory and store it in an instruction register;
  - Decode/execute the instruction.
- An extra step is included in the cycle to handle interrupts:
  - Sense the interrupt request line. If there's a signal on it, then respond to the interrupt.

Interrupts of CPU are prioritised and filtered



How the OS responds to an interrupt

When an interrupt is raised, the CPU switches to kernel mode, and then a handler (function) provided by the OS is invoked, so the control of the CPU is given to the OS. More specifically:

1. The CPU saves the computing *context* of the current process.
2. The CPU transfers control to a fixed memory location holding an appropriate interrupt handler (AKA ISR—Interrupt Service Routine).
3. When the routine is finished, the CPU restores the *context* of the interrupted process.

The interrupt vector table is an array of locations that hold the addresses (interrupt vectors) of these interrupt-handling routines. The OS sets up the initial interrupt vector table during booting.

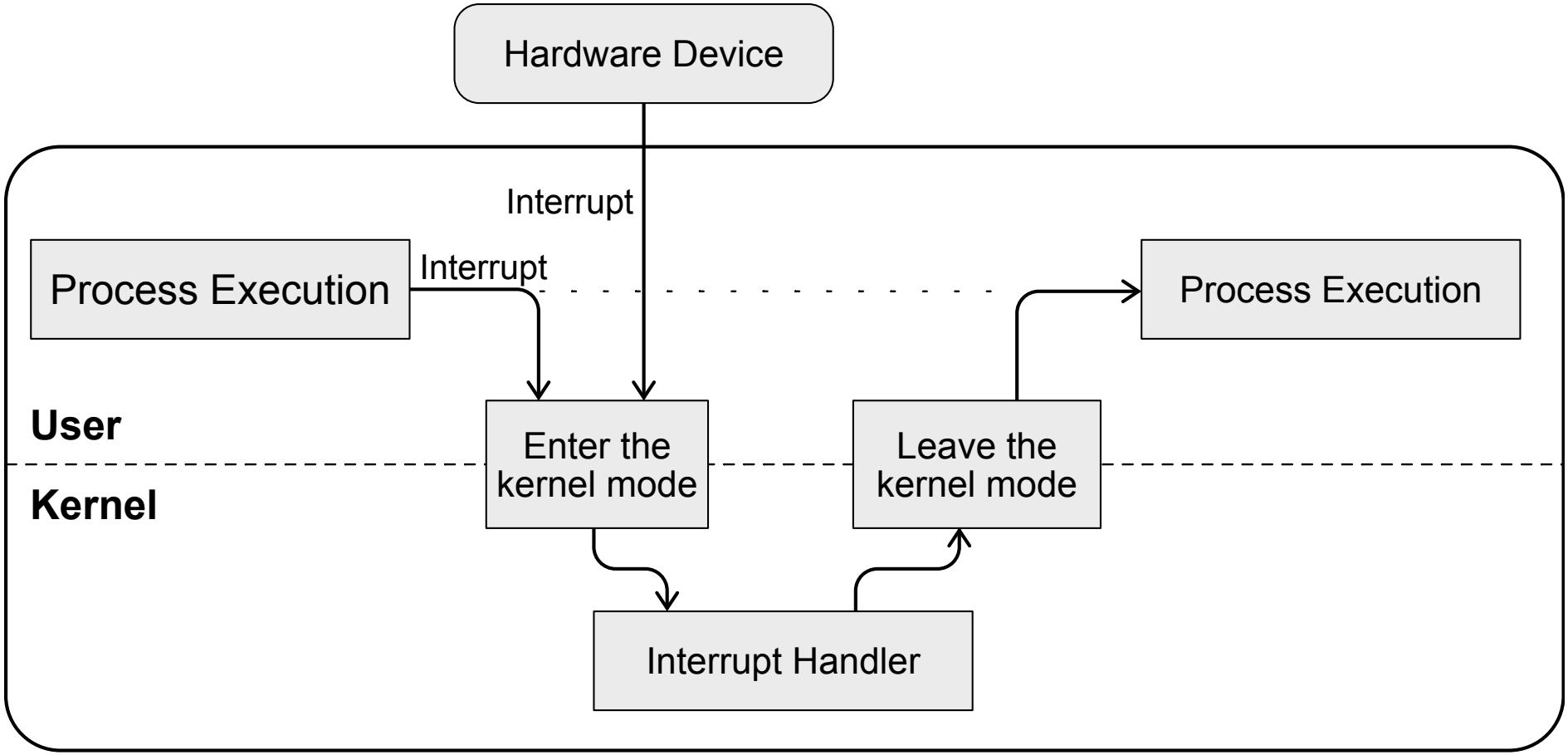
Different kinds of interrupts

CPU interrupts can be generated in three main ways:

- **Errors** e.g., division by zero; invalid memory access. These are also called exceptions.
- **Interrupt signals** from external devices: e.g., the completion of an input/output (I/O) operation requested by OS, or arrival of a network packet at the network interface card (NIC).
- **System calls** (also called ‘traps’ or ‘software interrupts’) that are caused by a particular CPU instruction (e.g., INT or SVC).

The OS kernel prepares a handler function for each of these interrupts.

Interrupt handling happens in kernel mode



How is a process created?

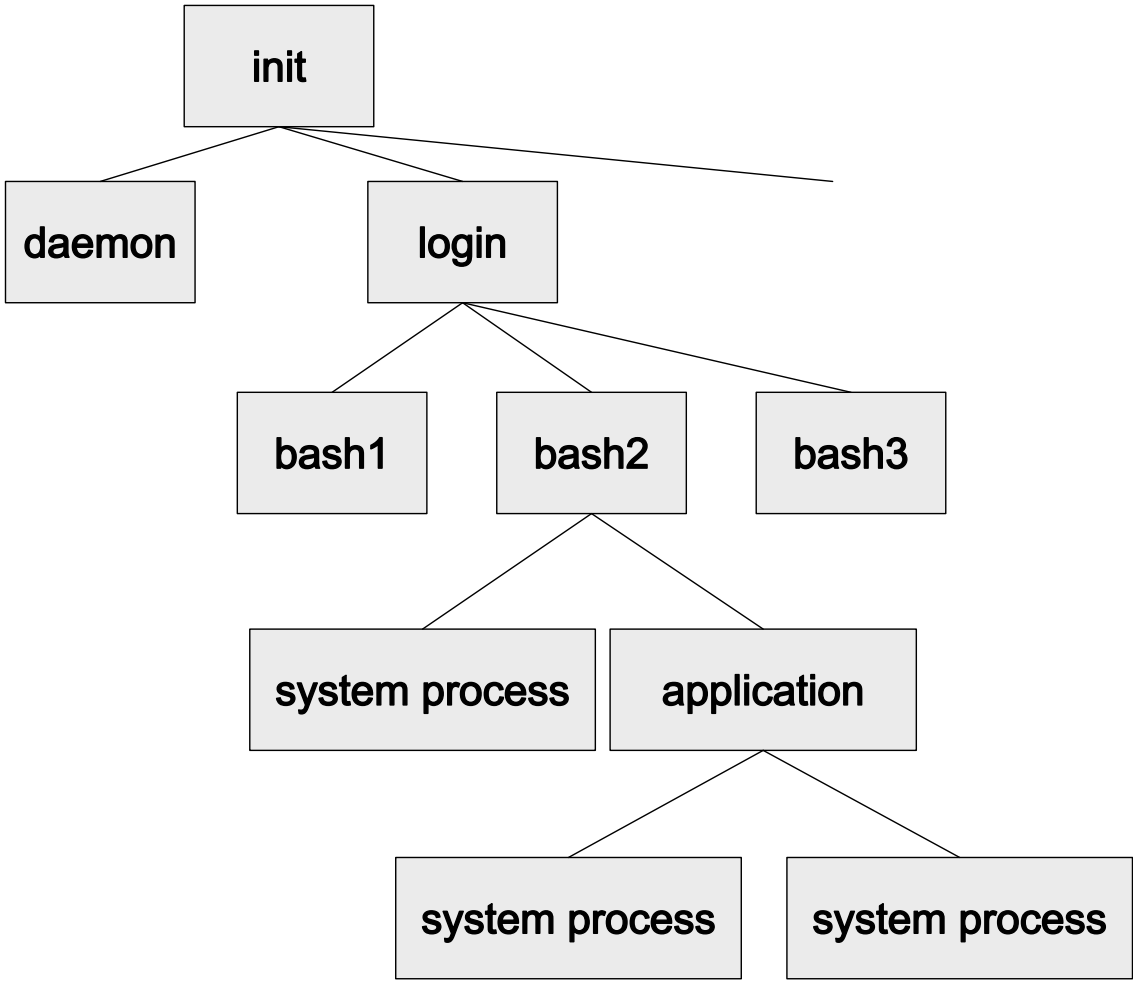
A process is created by another process: we talk of parent and child processes.

- A process can create a copy of itself using "`fork()`";
- The child process can then call "`exec()`" to load a new program;
  - A child process *can* run concurrently with its parent.
  - A child process *can* share the resources of its parent if not restricted.

  Creating processes is a very powerful functionality. The whole system starts with one process called `init`, which is manually created by the OS kernel.

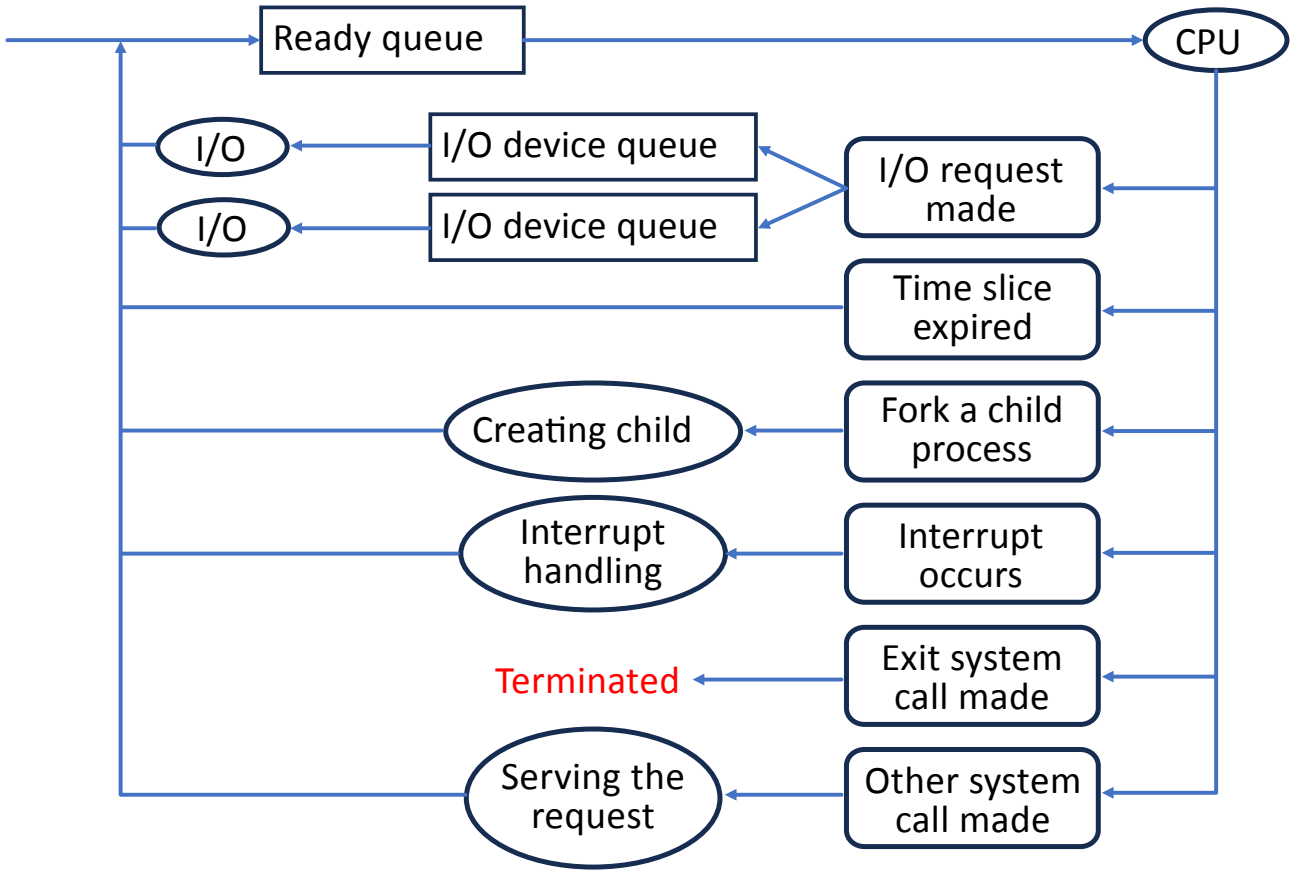
The process hierarchy

Processes form a tree:



Lifecycle of processes in an OS

Rounded rectangles are events (hardware or software triggered).
Rectangles are queues where processes wait.
Circles are processing resources that serve the events/queues.



Readings

For this lecture, relevant readings are Chapter 2 (Sections 1–7) and Chapter 3 (Sections 1–3) of Silberschatz *et al*.